



Lista de Exercícios 2

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 2 - Complexidade de Algoritmos

1. Encontre o número de maneiras diferentes de subir uma escada com n degraus se cada passo pode ter um ou dois degraus. Por exemplo, uma escada de três degraus pode ser escalada de três maneiras: 1-1-1, 1-2 e 2-1.

Solução: Considerando n o número de degraus da escada, temos que:

Para $n=1$:

É necessário dar apenas um passo, logo:

- 1 degrau

$\Rightarrow \text{Degrau}(1) = 1$ maneira.

Para $n=2$:

Podemos subir a escada de duas maneiras:

- 1 degrau

- 2 degraus

$\Rightarrow \text{Degrau}(2) = 2$ maneiras.

Para $n=3$:

Podemos subir a escada de três maneiras:

- 1 + 1 + 1 degraus

- 1 + 2 degraus

- 2 + 1 degraus

$\Rightarrow \text{Degrau}(3) = 3$ maneiras.

Para $n=4$:

Podemos subir a escada de cinco maneiras:

- 1 + 1 + 1 + 1 degraus

- 1 + 1 + 2 degraus
- 2 + 2 degraus
- 2 + 1 + 1 degraus
- 2 + 2 + 1 degraus

$\Rightarrow \text{Degrau}(4) = 5$ maneiras.

Para n=5

Podemos subir a escada de cinco maneiras:

- 1 + 1 + 1 + 1 + 1 degraus
- 1 + 1 + 1 + 2 degraus
- 1 + 1 + 2 + 1 degraus
- 1 + 2 + 1 + 1 degraus
- 1 + 2 + 2 degraus
- 2 + 1 + 1 + 1 degraus
- 2 + 1 + 2 degraus
- 2 + 2 + 1 degraus
- 2 + 2 + 1 degraus

$\Rightarrow \text{Degrau}(5) = 8$ maneiras.

Em resumo, obtemos as seguintes relações:

$\text{Degrau}(1) = 1$
 $\text{Degrau}(2) = 2$
 $\text{Degrau}(3) = 3$
 $\text{Degrau}(4) = 5$
 $\text{Degrau}(5) = 8$

Portanto, é possível notar que a função Degrau se assimila à sequência de Fibonacci

$$\Rightarrow \text{Desse modo, a função Degrau}(n) = \begin{cases} 1, & \text{se } n = 1 \\ 2, & \text{se } n = 2 \\ \text{Degrau}(n-1) + \text{Degrau}(n-2), & \text{se } n \geq 3 \end{cases}$$

2. Na linguagem de sua escolha, implemente dois algoritmos para calcular o enésimo número de Fibonacci: (a) o algoritmo baseado na definição recursiva e (b) o algoritmo iterativo. Plote um gráfico com o tempo de ambos os algoritmos para cada valor de $n = 1, \dots, n_{max}$. Em que n_{max} é o maior valor de n para o qual ambos programas executam em menos de 1 minuto em seu computador

Solução:

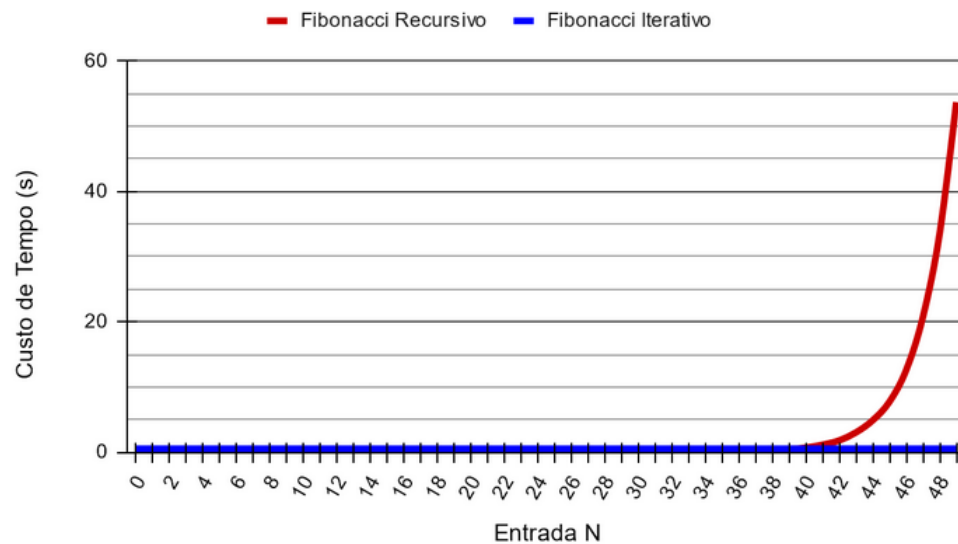
(a) Fibonacci Recursivo:

```
long fib_recursivo(long n){  
    if(n==0) return 0;  
    if(n==1) return 1;  
    return fib_recursivo(n-1)+fib_recursivo(n-2);  
}
```

(b) Fibonacci Iterativo:

```
long fib_iterativo(long n){  
    long a=0,b=1,c;  
    for(int i=1;i<=n;i++)  
    {  
        c = a+b;  
        a = b;  
        b = c;  
    }  
    return a;  
}
```

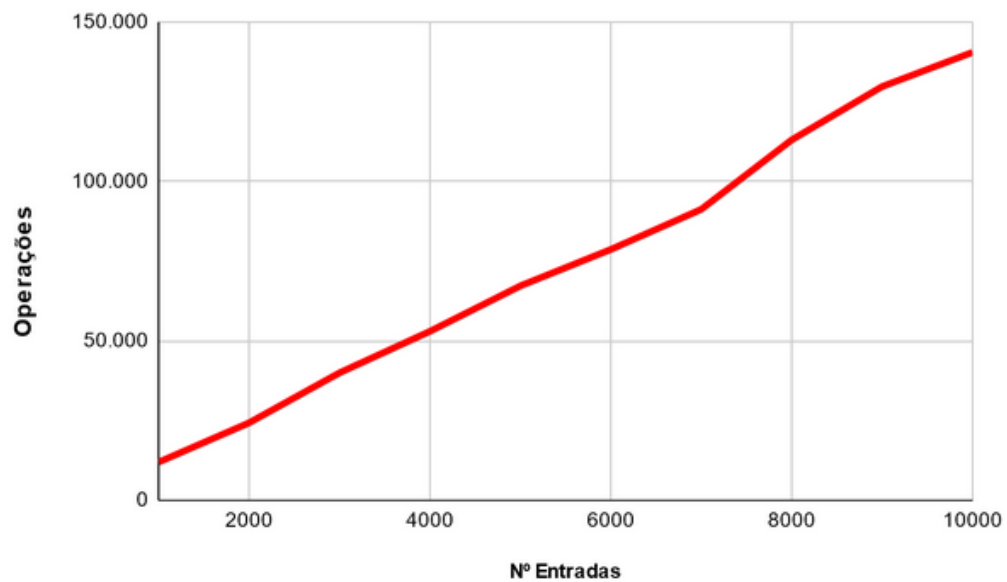
Gráfico Tempo x Entrada



3. Qual a provável complexidade de um algoritmo que possui as seguintes observações empíricas da contagem de sua operação básica:

size	count
1000	11.966
2000	24.303
3000	39.992
4000	53.010
5000	67.272
6000	78.692
7000	91.274
8000	113.063
9000	129.799
10000	140.538

Solução: A provável complexidade para esse algoritmo é linear, pois é possível observar que, de uma certa forma, os números de operações crescem linearmente com o aumento também linear do tamanho da entrada, como é possível ver no gráfico abaixo:



4. Em cada caso, indique se $f(n) = O(g(n))$ ou $f(n) = \Omega(g(n))$, ou ambos (neste caso $f(n) = \Theta(g(n))$):

	$f(n)$	$g(n)$
(1)	$n - 10$	$n - 200$
(2)	$n^{1/2}$	$n^{2/3}$
(3)	$100n + \log n$	$n + (\log n)^2$
(4)	$\log 2n$	$\log 3n$
(5)	$10 \log n$	$\log(n^2)$
(6)	$n^{1,01}$	$n \log^2 n$
(7)	$n^2 / \log n$	$n(\log n)^2$
(8)	$n^{0,1}$	$(\log n)^{10}$
(9)	$(\log n)^{\log n}$	$n / \log n$
(10)	$\sqrt{n}$	$(\log n)^3$
(11)	$n^{1/2}$	$5^{\log_2 n}$
(12)	$n2^n$	3^n
(13)	2^n	2^{n+1}
(14)	$n!$	2^n
(15)	$(\log n)^{\log n}$	$2^{(\log_2 n)^2}$
(16)	$\sum_{i=1}^n i^k$	n^{k+1}

Solução:

1. $f(n) = \Theta(g(n))$
2. $f(n) = O(g(n))$
3. $f(n) = \Theta(g(n))$
4. $f(n) = O(g(n))$
5. $f(n) = \Theta(g(n))$
6. $f(n) = O(g(n))$
7. $f(n) = \Omega(g(n))$
8. $f(n) = \Omega(g(n))$
9. $f(n) = O(g(n))$
10. $f(n) = \Omega(g(n))$
11. $f(n) = O(g(n))$

$$12. f(n) = O(g(n))$$

$$13. f(n) = \Theta(g(n))$$

$$14. f(n) = \Omega(g(n))$$

$$15. f(n) = O(g(n))$$

$$16. f(n) = O(g(n))$$

5. **Resolva a recorrência:**

$$T(n) = \begin{cases} 2, & \text{se } n = 2 \\ 2 \cdot T(\frac{n}{2}) + n, & \text{se } n = 2^k, \text{ para } k > 1 \end{cases}$$

Solução:

Desenvolvendo a recorrência obtemos as seguintes funções:

$$T(n) = 2 \cdot T(\frac{n}{2}) + n \quad (1)$$

$$T(n/2) = 2 \cdot T(\frac{n}{4}) + n/2 \quad (2)$$

$$T(n/4) = 2 \cdot T(\frac{n}{8}) + n/4 \quad (3)$$

Sabendo disso, temos que a partir de (1) e (2):

$$T(n) = 2 \cdot \underline{T(\frac{n}{2})} + n$$

$$T(n) = 2 \cdot (2 \cdot T(\frac{n}{4}) + \frac{n}{2}) + n$$

$$T(n) = 2^2 \cdot T(\frac{n}{4}) + 2n \quad (4)$$

De (3) e (4):

$$T(n) = 2^2 \cdot \underline{T(\frac{n}{4})} + 2n$$

$$T(n) = 2^2 \cdot (2 \cdot T(\frac{n}{8}) + \frac{n}{4}) + 2n$$

$$T(n) = 2^3 \cdot T(\frac{n}{8}) + 3n$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$\Rightarrow T(n) = 2^k \cdot T(\frac{n}{2^k}) + kn$$

A recorrência encerra, desse modo, quando:

$$\frac{n}{2^k} = 2 \Rightarrow 2^{k+1} = n \Rightarrow k+1 = \log_2(n) \Rightarrow k = \log_2(n) - 1$$

$$\Rightarrow k = \log_2(n) - \log_2(2) \Rightarrow k = \log_2(\frac{n}{2})$$

$$\Rightarrow T(n) = \frac{n}{2} \cdot T(2) + \log_2(\frac{n}{2}) \cdot n$$

$$\begin{aligned} \Rightarrow T(n) &= n + \log_2\left(\frac{n}{2}\right) \cdot n \Rightarrow T(n) = n + (\log_2(n) - \log_2(2)) \cdot n \\ \Rightarrow T(n) &= n - n + n \cdot \log_2(n) \quad \therefore T(n) = n \cdot \log_2(n) \end{aligned}$$

6. Descreva um algoritmo de tempo $\Theta(n \lg n)$ que, dado um conjunto S de n inteiros e um outro inteiro x , determine se existe ou não dois elementos em S cuja soma seja exatamente x .

Solução:

```
int verifySum(int x, int *S){
    mergeSort(S);
    int i = 0;
    int j = size(S) - 1;

    while(i < j){
        if(S[i] + S[j] > x)
            j--;
        else
            if (S[i] + S[j] < x)
                i++;
            else return 1;
    }
    return 0;
}
```

Nesse algoritmo, podemos observar a função `verifySum`, que recebe como parâmetros o inteiro x e o conjunto S de inteiros. Inicialmente, ordenamos o conjunto S através do algoritmo Merge Sort, cuja complexidade é $\Theta(n \lg n)$. Em seguida, presumimos que `size(S)` retorna n , visto que é o tamanho de S , e percorremos o conjunto em busca de dois inteiros nele que somando resulte no valor de x . As operações que sucedem o Merge Sort tem complexidade $\Theta(n)$. Portanto, conclui-se que o algoritmo acima possui complexidade dominante $\Theta(n \lg n)$.

7. **Resolva a recorrência:** $a_n = a_0 + a_1 + \dots + a_{n-1}$, em que $a_0 = 1$.

Solução: Para resolvermos a recorrência e encontrarmos sua fórmula fechada, devemos expandi-la de forma a encontrarmos um padrão. Dessa forma:

Se $n = 0$, $a_0 = 1$;

Se $n = 1$, $a_1 = a_0 = 1 = 2^0$;

Se $n = 2$, $a_2 = a_0 + a_1 = 1 + 1 = 2 = 2^1$

Se $n = 3$, $a_3 = a_0 + a_1 + a_2 = 1 + 1 + 2 = 4 = 2^2$

Se $n = 4$, $a_4 = a_0 + a_1 + a_2 + a_3 = 1 + 1 + 2 + 4 = 8 = 2^3$
 Se $n = 5$, $a_5 = a_0 + a_1 + a_2 + a_3 + a_4 = 1 + 1 + 2 + 4 + 8 = 16 = 2^4$
 $\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$
 Portanto, se $n = i$, $a_i = a_0 + a_1 + a_2 + a_3 + a_4 + \dots a_{i-1} = 2^{i-1}$

Sendo assim, conclui-se através do padrão observado que para $i = n$:

$$a_n = a_0 + a_1 + a_2 + a_3 + a_4 + \dots + a_{n-1} = 2^{n-1}$$

Logo, a solução da recorrência é 2^{n-1} .

8. Considere o seguinte trecho de código:

```

1: procedure PROG1(inteiro positivo  $k$ )
2:   if  $k == 1$  then
3:     Print("Oi") return
4:   else
5:     for  $i \leftarrow 1 \rightarrow k$  do
6:       Print("Oi")
7:     end for
8:     Prog1( $k - 1$ )
9:   end if
10: end procedure

```

Seja $T(n)$ o número de vezes que a palavra OI é impressa quando *Prog1* é chamado com parâmetro n . Calcule $T(1)$ e determine uma relação entre $T(n)$ e $T(n - 1)$ para $n > 1$. Com base na relação encontrada, utilize indução para mostrar que $T(n) \leq n^2$, para todo n maior ou igual a 1.

Solução:

Para $n = 1$, a palavra "OI" é impressa apenas uma vez, portanto, $T(1) = 1$.

Para $n > 1$, a palavra "OI" é inicialmente impressa n vezes, através do *loop for*. Em seguida, ela é impressa $T(n-1)$ vezes. Portanto, $T(n) = n + T(n-1)$.

Sendo assim,

$$T(n) = \begin{cases} 1, & \text{se } n = 1 \\ n + T(n - 1), & \text{se } n > 1 \end{cases}$$

Dando continuidade, devemos mostrar que $T(n) \leq n^2$, para todo n maior ou igual a 1. Por indução, temos:

Passo base ($k = 1$):

Neste caso, $T(1) = 1 \leq 1^2$. $\therefore P(1)$ é verdadeiro

Hipótese de Indução ($k > 1$):

$T(k) \leq k^2$, para $k > 1$.

Passo Indutivo:

No passo indutivo, aceitamos a hipótese de indução $P(k)$, para $k > 1$ e, portanto, precisamos provar por indução que se $P(k + 1)$ é verdadeiro, $P(k)$ também é.

$$T(k + 1) = k + 1 + T(k).$$

Como a hipótese $P(k)$ garante que $T(k) \leq k^2$:

$$T(k + 1) \leq k + 1 + k^2 < (k + 1)^2$$

$\therefore P(k + 1)$ é verdadeiro. $\Rightarrow P(k)$ é verdadeiro.



9. Considere o seguinte algoritmo recursivo, cujo argumento n é um inteiro positivo.

```
1: procedure ASTERISCO( $n$ )
2:   if  $n > 0$  then
3:     ASTERISCO( $n - 1$ )
4:     for  $i \leftarrow 1 \rightarrow n$  do
5:       imprima "*"
6:     end for
7:     ASTERISCO( $n - 1$ )
8:   end if
9: end procedure
```

Para um dado valor de n , quantos asteriscos serão impressos em uma chamada de $ASTERISCO(n)$?

Solução:

Considerando $A(n)$ como a quantidade de asteriscos impressos dado um valor n , temos que:

$$A(n) = \begin{cases} 0, & \text{se } n = 0 \\ 2^{n+1} - (n + 2), & \text{se } n \geq 1 \end{cases}$$

Como exemplo, temos:

Para $n = 1$, $2^2 - (1+2) = 4 - 3 = 1$ asterisco impresso;
 Para $n = 2$, $2^3 - (2+2) = 8 - 4 = 4$ asteriscos impressos;
 Para $n = 3$, $2^4 - (3+2) = 16 - 5 = 11$ asteriscos impressos;
 Para $n = 4$, $2^5 - (4+2) = 32 - 6 = 26$ asteriscos impressos;
 $\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$
 Para $n = i$, $2^{i+1} - (i+2)$ asteriscos impressos.

10. **Mostre que:**

- (a) **A solução de $T(n) = T(n - 1) + n$; $T(1) = 1$ é $O(n^2)$.**
- (b) **A solução de $T(n) = T(\lceil n/2 \rceil) + 1$; $T(1) = 1$ é $O(\lg n)$.**
- (c) **A solução de $2T(\lfloor n/2 \rfloor) + n$; $T(1) = 1$ é $O(n \cdot \lg n)$.**

Solução:

- (a) **Solução:** Desenvolvendo a recorrência, temos que:

$$T(n) = T(n - 1) + n \quad (1)$$

$$T(n - 1) = T(n - 2) + n - 1 \quad (2)$$

$$T(n - 2) = T(n - 3) + n - 2 \quad (3)$$

De (1) e (2):

$$T(n) = T(n - 2) + (n - 1) + n \quad (4)$$

De (3) e (4):

$$T(n) = T(n - 3) + (n - 2) + (n - 1) + n \quad (4)$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$T(n) = T(n - k) + (n - k + 1) + (n - k + 2) + \dots + (n - 2) + (n - 1) + n$$

Neste caso, o algoritmo encerra quando $n - k = 1 \Rightarrow k = n - 1$

$$T(n) = T(n - (n - 1)) + (n - (n - 1) + 1) + (n - (n - 1) + 2) + \dots + (n - 2) + (n - 1) + n$$

$$T(n) = T(1) + (2) + (3) + \dots + (n - 2) + (n - 1) + n$$

$$T(n) = 1 + (2) + (3) + \dots + (n - 2) + (n - 1) + n$$

$$\Rightarrow T(n) = \frac{n \cdot (n + 1)}{2}$$

Para mostrar que $T(n)$ é $O(n^2)$ basta definir $f(n) = \frac{n \cdot (n + 1)}{2}$ e $g(n) = n^2$, sendo assim basta encontrar n_0 e c , tais que $f(n) < c \cdot g(n)$, $\forall n \geq n_0$, neste sentido, basta tomar $n_0 = 1$ e $c = 3$, por exemplo. Neste caso, vemos que $T(n) = O(n^2)$.

(b) **Solução:** Supondo que n é uma potência de 2, ou seja $n = 2^k$, segue-se que desenvolvendo a recorrência:

$$T(n) = T(n/2) + 1 \quad (1)$$

$$T(n/2) = T(n/4) + 1 \quad (2)$$

$$T(n/4) = T(n/8) + 1 \quad (3)$$

Sabendo disso, temos que a partir de (1) e (2): $T(n) = \underline{T(n/2)} + 1$

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/4) + 2 \quad (4)$$

De (3) e (4): $T(n) = \underline{T(n/8)} + 2$

$$T(n) = T(n/8) + 1 + 2$$

$$T(n) = T(n/8) + 3$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$T(n) = T(n/2^k) + k$$

A recorrência encerra, desse modo, quando:

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2(n)$$

$$\Rightarrow T(n) = T(1) + \log_2 n$$

$$\Rightarrow T(n) = 1 + \log_2(n)$$

Para mostrar que $T(n)$ é $O(\log n)$ basta definir $f(n) = 1 + \log(n)$ e $g(n) = \log n$ e encontrar n_0 e c , tais que $f(n) < c \cdot g(n)$, $\forall n \geq n_0$. Sendo assim, tomamos $n_0 = 2$ e $c = 3$ e provamos que $T(n) = O(\log_2 n)$.

(c) **Solução:** Supondo que n é uma potência de 2, ou seja $n = 2^k$, segue-se que:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n \quad (1)$$

$$T(n/2) = 2 \cdot T\left(\frac{n}{4}\right) + n/2 \quad (2)$$

$$T(n/4) = 2 \cdot T\left(\frac{n}{8}\right) + n/4 \quad (3)$$

Sabendo disso, temos que a partir de (1) e (2):

$$T(n) = 2 \cdot \underline{T\left(\frac{n}{2}\right)} + n$$

$$T(n) = 2 \cdot \left(2 \cdot T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n$$

$$T(n) = 2^2 \cdot T\left(\frac{n}{4}\right) + 2n \quad (4)$$

De (3) e (4):

$$T(n) = 2^2 \cdot \underline{T\left(\frac{n}{4}\right)} + 2n$$

$$T(n) = 2^2 \cdot \left(2 \cdot T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + 2n$$

$$T(n) = 2^3 \cdot T\left(\frac{n}{8}\right) + 3n$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$\Rightarrow T(n) = 2^k \cdot T\left(\frac{n}{2^k}\right) + kn$$

A recorrência encerra, desse modo, quando:

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2(n)$$

$$\Rightarrow T(n) = n \cdot T(1) + \log_2 n \cdot n$$

$$\Rightarrow T(n) = n + n \cdot \log_2(n)$$

Para mostrar que $T(n)$ é $O(n \cdot \log_2 n)$ basta definir $f(n) = n + n \cdot \log_2 n$ e $g(n) = n \cdot \log_2 n$, sendo assim basta encontrar n_0 e c , tais que $f(n) < c \cdot g(n)$, $\forall n \geq n_0$, neste sentido, basta tomar $n_0 = 2$ e $c = 3$, por exemplo. Neste caso, é possível notar que $T(n) = O(n \cdot \log_2 n)$.